

Notes (Week 8 Monday)

These are the notes I wrote during the lecture.

We introduced the toy programming language $\mathcal{L}$.

We use P, Q to range over programs, $P \in \mathcal{L}$

The P programs are inductively defined by this grammar:

```
P ::=  x := e      (assignment)
      |  P; Q      (seq)
      |  if b then P else Q fi  (if)
      |  while b do P od      (while)
```

^ this is called a BNF (Backus-Naur Form) grammar

The syntax is defined by a bunch of clauses separated by |

Each clause is one clause of an inductive definition

| $P; Q$ means

If P is a program, and Q is a program, then $P; Q$

But it connects to the language of predicate logic as follows:

```
in  x:=e    e is a term over some vocabulary
    if b ...    b is a predicate over the same vocabulary
    while b ...
```

But mostly, we'll assume a vocabulary of integer operations and predicates on integers with their standard meaning
+, -, *, ≤, ...

If you want programs that have more stuff than just ints, (arrays, floats, ...) just chuck it in the vocab

$\{\phi\} P \{\psi\}$

ϕ the precondition is a predicate about states

P the program $P \in \mathcal{L}$

ψ the postcondition is a predicate states

The Hoare triple means:

If ϕ is true in the initial state,
and we run P from that state,
then if we terminate,
 ψ holds in the final state

Two questions:

1. How do you prove a Hoare triple?
2. What does a Hoare triple mean? (formally)

Notice the symmetry with (predicate) logic.

1. How do you prove a formula to be valid? (natural deduction)
2. What do formulas mean? (models, interpretations, $\models$, ..)

$\vdash$ "provability"
 $\models$ "truth"

$\vdash \phi$ " ϕ is provable using natural deduction"
 $\models \phi$ " ϕ is true in every possible model+interpretation"

To prove a Hoare triple, we use the inference rules of Hoare logic.

$\vdash \{\phi\} P \{\psi\}$ "this Hoare triple is provable using the inference rules of Hoare logic"
 $\models \{\phi\} P \{\psi\}$ "this Hoare triple is true about every possible execution of P"

We make this distinction for two reasons:

- It might not be the case that Hoare logic is powerful enough to prove everything that's true
- It might be that Hoare logic is defined so that it proves false things.

$$\frac{A \quad B}{A \wedge B} \text{ (conjI)}$$

If I have a proof of A, and a proof of B,
 I can invoke conjI to get a proof of $A \wedge B$

The (long-awaited, long-teased) inference rules!!

```
// 3 = z
x := 3
// x = z
```

$$\frac{}{\{\phi[x:=e]\} \ x:=e \ \{\phi\}} \text{ (assign)}$$

```
// 3 = 3
z := 3;
// 3 = z
x := 3
// x = z
```

$$\frac{\frac{}{\{3 = 3\} \ z:=3 \ \{3 = z\}} \text{ (assign)} \quad \frac{}{\{3 = z\} \ x:=3 \ \{x = z\}} \text{ (assign)}}{\{3 = 3\} \ z:=3; \ x:=3 \ \{x = z\}} \text{ (seq)}$$

$\{\phi\} P \{\rho\} \quad \{\rho\} Q \{\psi\}$

$$\frac{}{\{\phi\} P; Q \{\psi\}} \text{ (seq)}$$

We're going to execute either P or Q (well, duh)
But we don't necessarily know which one.

ϕ might not imply either b or $\neg b$

$$\frac{\{\phi \wedge b\} P \{\psi\} \quad \{\phi \wedge \neg b\} Q \{\psi\}}{\{\phi\} \text{ if } b \text{ then } P \text{ else } Q \text{ fi } \{\psi\}} \text{ (if)}$$

issue: if $\phi = (x=7)$,
then $\phi[x:=y] = 7=y$

the assignment rule lets us prove

$\{y=7\} x:=y \{x=7\}$

$y=7$ and $\top \wedge y = 7$ are different (but equivalent) formulas

What we need is the ability to "massage" pre and postconditions
to fit into other rules: rule of consequence

$$\frac{\frac{}{\{y=7\} x:=y \{x=7\}} \text{ (assign)}}{\{\top \wedge y = 7\} x := y \{x=7\}} \text{ (conseq)} \quad \frac{\frac{}{\{7=7\} x:=7 \{x=7\}} \text{ (assign)}}{\{\top \wedge y \neq 7\} x:=7 \{x=7\}} \text{ (conseq)}$$

$$\frac{}{\{\top\} \text{ if } y = 7 \text{ then } x := y \text{ else } x:=7 \{x=7\}} \text{ (if)}$$

- We can predict *exactly* what a program will do without running it
- Even though there's infinitely many possible states, we only have to do the argument once (c.f. testing)
- The real win comes from the while rule

$$\frac{\{b \wedge \phi\} P \{\phi\}}{\{\phi\} \text{ while } b \text{ do } P \text{ od } \{\neg b \wedge \phi\}} \text{ (while)}$$

ϕ is called a *loop invariant*.

A loop invariant is:

- true in the initial state
- if it's true before a loop iteration, then it's true after the loop iteration

Intuition is:

if something is true initially,
and running the loop doesn't falsify it,
then it must be true after the last loop iteration

// we want to compute $X*Y$, where x,y are positive
// we're going to pretend we don't have a multiplication operator

```

{⌈}
{x + X*(Y-0) = X*Y}
x := 0;
{x + X*(Y-0) = X*Y}
y := 0;
{x + X*(Y-y) = X*Y}
while y ≠ Y do
  {y ≠ Y ∧ x + X*(Y-y) = X*Y}
  {x + X*(Y-y) = X*Y}
  {x+x + X*(Y-(y+1)) = X*Y}
  x := x+x;
  {x + X*(Y-(y+1)) = X*Y}
  y := y+1
  {x + X*(Y-y) = X*Y}
od
{y = Y ∧ x + X*(Y-y) = X*Y}
{x=X*Y}

```

$$\frac{\frac{\frac{\{ \top \} \ x=0; \ y=0 \ \{ x + X*(Y-y) = X*Y \}}{\text{.....}} \quad \frac{\text{.....}}{\text{(while)}} \quad \frac{\text{.....}}{\text{(conseq)}} \quad \frac{\{ x + X*(Y-y) = X*Y \} \text{ while } y \neq Y \text{ do } \dots \text{ od } \{ x=X*Y \}}{\text{(seq once)}}}{\{ \top \} \ x=0; \ y=0; \text{ while } y \neq Y \text{ do } \dots \text{ od } \{ x=X*Y \}}$$

- In Hoare logic, no matter how many loop iterations the program runs, we only reason about the loop body once

$$\frac{\varphi \Rightarrow \varphi' \quad \{\varphi'\} P \quad \{\Psi'\} \quad \Psi' \Rightarrow \Psi}{\{\varphi\} P \quad \{\psi\}} \quad (\text{conseq})$$

```
{x=2} P {x is odd}
^ in order to prove that, it should suffice to prove
{x is even} P {x=3}
```

If I've proven a Hoare triple,
this should imply similar Hoare triples
with more specific preconditions
and more vague postconditions

Now we know how to prove things in Hoare logic.

$$\vdash \{ \phi \} P \{ \psi \}$$

means "is provable by the inference rules of Hoare logic"

$$\models \{\phi\} \text{ P } \{\psi\}$$

```
means "the Hoare triple holds about every possible execution
      of the program"
```

To define $\models$, we need to give a semantics to $\mathcal{L}$.

Use S to denote the set of all states.

1. Define a transition system to describe how $\mathcal{L}$ programs execute step-by-step (operational semantics)

- For convenience, assume "skip" is a primitive in $\mathcal{L}$
- Define an unlabelled transition system, where
 - the set of states is $S \times \mathcal{L}$
 - the transitions are defined inductively by a bunch of inference rules:

$\llbracket e \rrbracket s$ "the value of the expression e in state s "
 $s[x:=v]$ means "the state s , but with the value of x updated to be v "

$$\frac{}{(s, x:=e) \rightarrow (s[x:=\llbracket e \rrbracket s], \text{skip})}$$

$$(s, P) \rightarrow (s', P')$$

$$\frac{}{(s, P; Q) \rightarrow (s', P; Q)}$$

$$\frac{}{(s, \text{skip}; Q) \rightarrow (s, Q)}$$

$$\llbracket b \rrbracket s = \text{false}$$

$$\frac{}{(s, \text{while } b \text{ do } P \text{ od}) \rightarrow (s, \text{skip})}$$

$$\llbracket b \rrbracket s = \text{true}$$

$$\frac{}{(s, \text{while } b \text{ do } P \text{ od}) \rightarrow (s, P; \text{while } b \text{ do } P \text{ od})}$$

...similar rules for if

Now, we can use this transition system to define $\models$

$\models \{\phi\} P \{\psi\}$ is true iff

For all state s and s' such that:

$\llbracket \phi \rrbracket s = \text{true}$, and
 $(s, P) \rightarrow^* (s', \text{skip})$,
 then $\llbracket \psi \rrbracket s' = \text{true}$

2. Define a function $\llbracket \cdot \rrbracket$ which maps $\mathcal{L}$ programs to relations between initial and final states.
 (denotational semantics)

$$\llbracket \cdot \rrbracket : \mathcal{L} \rightarrow (S \times S)$$

$(s, s') \in \llbracket P \rrbracket$ this means that running P from s can result in final state s'

$$\llbracket x := e \rrbracket = \{(s, s') \mid s' = s[x:=\llbracket e \rrbracket s]\}$$

```

[[P; Q]] = [[P]]; [[Q]]
[[if b then P else Q fi]] =
  ((b); [[P]]) ∪ ((¬b); [[Q]])
[[while b do P od]]
  ((b); [[P]])*; (¬b)

```

$\models \{\phi\} P \{\psi\}$ holds iff
 Whenever $(s, s') \in [[P]]$ and $\models \phi s$,
 then $\models \psi s'$

It doesn't matter whether we define $\models$ in terms of the denotational or operational semantics, they are both equivalent.

Here $;$ is overloaded. It's sequential composition on programs, and it's also **relational composition** on relations

For if, the following will help:

```

(b) = {(s, s) | [[b]]s = true}
(b), intuitively means
  "do nothing, but abort if b is false"

```

A **state** is a mapping from variable names to values.

– Hoare logic is sound

$\vdash \{\phi\} P \{\psi\}$ implies $\models \{\phi\} P \{\psi\}$

– Hoare logic is complete-ish

The rule of consequence might not be powerful enough
 to prove all implications (Gödel's fault)
 Therefore, Hoare logic is incomplete on a technicality

If we pretend that all valid implications are provable,
 then it's complete (relative completeness)